
GRAFIKA KOMPUTEROWA - LAB 09

Table of Contents

Joanna Dagil 231008	1
CEL CWICZENIA	1
ZADANIA	1
ZADANIE 1	2
ZADANIE 2	4
ZADANIE 3	6
ZADANIE 4	8
ZADANIE 5	10
ZADANIE 6	12
FUNKCJE POMOCNICZE	12
LAMBERT	12
PHONG	13
CONST	14
GOURAUD	15
GOURAUD SPEC	17
PHONG SPEC	18
AVE VECT	20
Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie	21
Funkcja rysuje odcinek na obrazie monochromatycznym.	21
Funkcja do wyznaczania parametrów płaszczyzny $Ax+By+Cz+D=0$ przechodzącej przez trzy punkty p_1, p_2, p_3 ...	22
Funkcja do wyznaczania odległości punktu $[x, y, z]$ od płaszczyzny o równaniu $Ax+By+Cz+D=0$	23
Funkcja wypełnia zamknięty obszar obrazu metoda floodfill.	26

Joanna Dagil 231008

```
clear; close all; clc;
```

CEL CWICZENIA

Rozszerzenie modeli oświetlenia i cieniowania.

ZADANIA

% Dane są cztery trójkąty (a dokładniej cztery ściany boczne ostrosłupa o podstawie kwadratowej) o następujących współrzędnych narożników:

% Trójkąt 1:

```
tri(1).A = [ 0; 0; 10];
```

```
tri(1).B = [20; 20; 20];
```

```
tri(1).C = [20; -20; 20];
```

% Trójkąt 2:

```
tri(2).A = tri(1).A;
```

```
tri(2).B = tri(1).C;
```

```
tri(2).C = [-20; -20; 20];
```

% Trójkąt 3:

```
tri(3).A = tri(1).A;
tri(3).B = tri(2).C;
tri(3).C = [-20; 20; 20];
% Trójkąt 4:
tri(4).A = tri(1).A;
tri(4).B = tri(3).C;
tri(4).C = tri(1).B;
% Zwróć uwagę, że w sumie widzimy pięć różnych narożników.

% Wszystkie trójkąty mają współczynnik odbicia
e = 1;

% Scena ta wyświetlana jest na obrazie uzyskanym przez rzutowanie równoległe
na płaszczyznę XY i zdyskretyzowanym do rozdzielczości XxY = 640x480
X = 640;
Y = 480;

% przy czym rozmiar pojedynczego pixela to 0.1x0.1.
dx = 0.1;
dy = 0.1;

% Oświetlenie tła to
amb = 20;

% Dodatkowo scena oświetlona jest pojedynczym punktowym źródłem światła
umieszczonym w
lamp = [10; 0; 0];
% Natężenie źródła światła wynosi
int = 235;
% a spadek natężenia oświetlenia w zależności od odległości wynosi:
%  $\text{int} / (1 + 0.001 || \mathbf{p} - \mathbf{lamp} ||^2)$ 

% Jeśli potrzebne jest położenie „oka obserwatora”, to przyjmij
eye = [65; 0; 0];
```

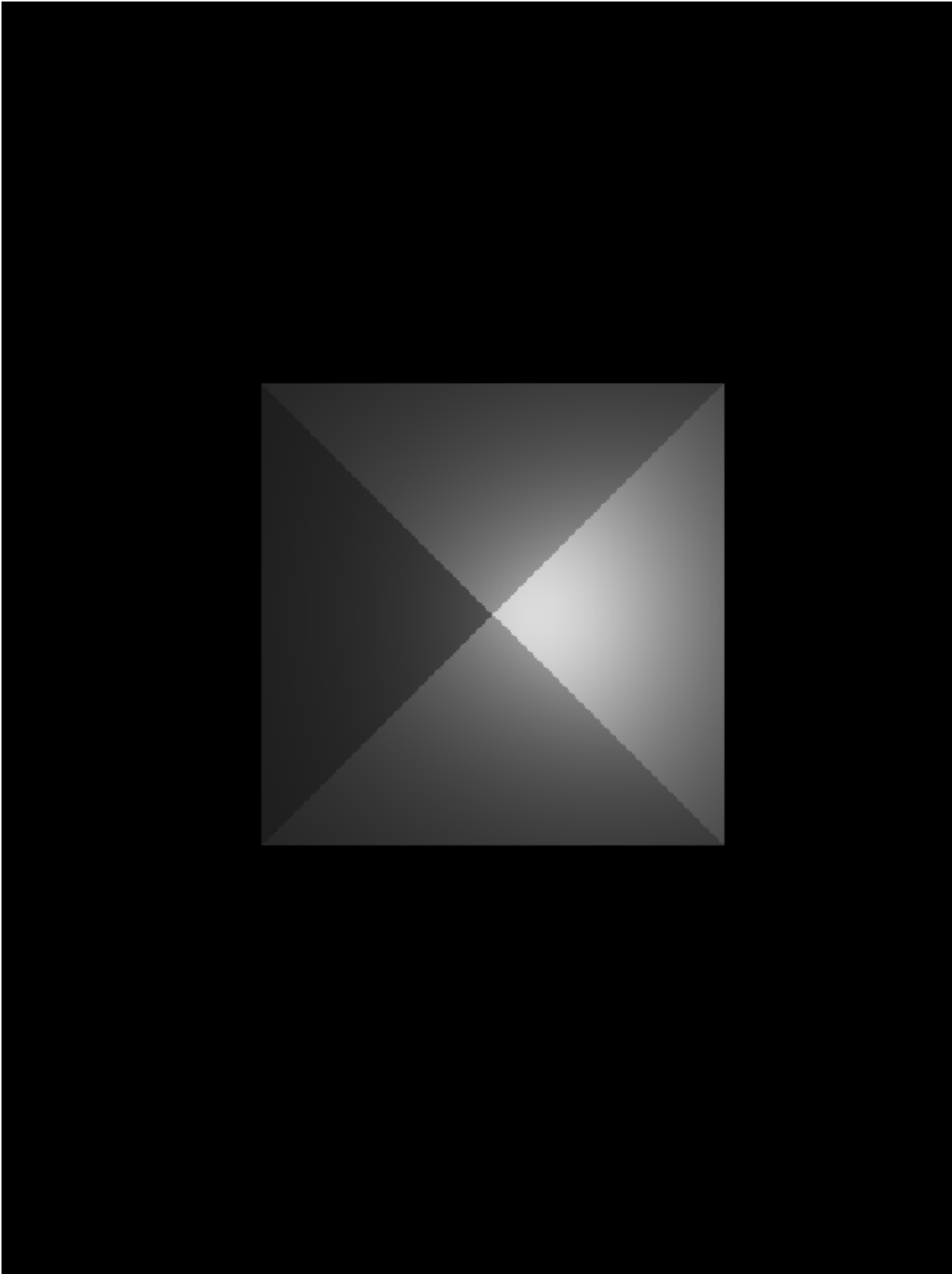
ZADANIE 1

Używając modelu oświetlenia Lamberta, wygeneruj obraz tej sceny.

```
Im = uint8(zeros(Y,X));

Im = lambert(Im, tri(1), amb, lamp, int, X, Y, dx, dy);
Im = lambert(Im, tri(2), amb, lamp, int, X, Y, dx, dy);
Im = lambert(Im, tri(3), amb, lamp, int, X, Y, dx, dy);
Im = lambert(Im, tri(4), amb, lamp, int, X, Y, dx, dy);

figure(1); imshow(Im);
```



ZADANIE 2

Używając modelu oświetlenia Phong, wygeneruj obraz tej sceny. Przykładowa wartość parametru m wynosi 15.

```
m = 15;
```

```
Im = uint8(zeros(Y,X));
```

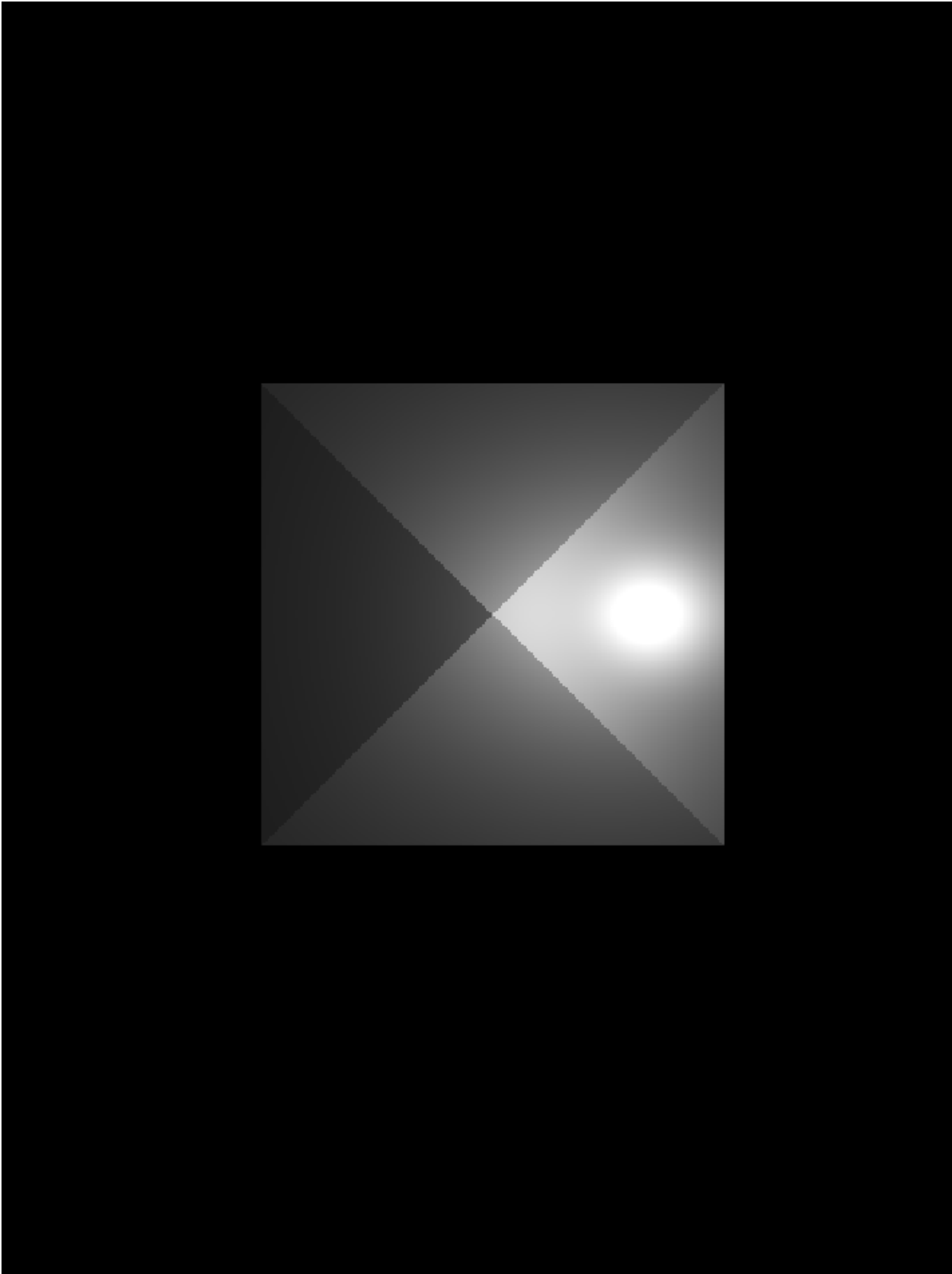
```
Im = phong(Im, tri(1), amb, lamp, int, X, Y, dx, dy, e, eye, m);
```

```
Im = phong(Im, tri(2), amb, lamp, int, X, Y, dx, dy, e, eye, m);
```

```
Im = phong(Im, tri(3), amb, lamp, int, X, Y, dx, dy, e, eye, m);
```

```
Im = phong(Im, tri(4), amb, lamp, int, X, Y, dx, dy, e, eye, m);
```

```
figure(2); imshow(Im);
```



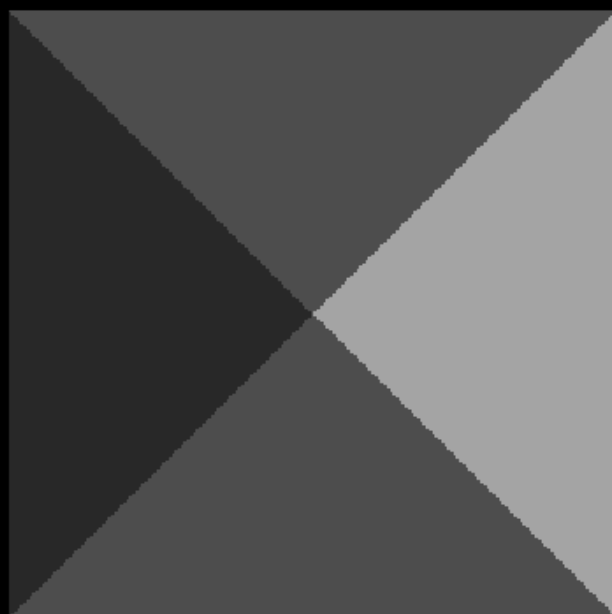
ZADANIE 3

Wygeneruj obraz tej sceny używając cieniowania stałego (np. na podstawie środków poszczególnych trójkątów). Do obliczania oświetlenia wybranych punktów zastosuj model Lamberta.

```
Im = uint8(zeros(Y,X));

Im = const(Im, tri(1), amb, lamp, int, X, Y, dx, dy);
Im = const(Im, tri(2), amb, lamp, int, X, Y, dx, dy);
Im = const(Im, tri(3), amb, lamp, int, X, Y, dx, dy);
Im = const(Im, tri(4), amb, lamp, int, X, Y, dx, dy);

figure(3); imshow(Im);
```



ZADANIE 4

Wygeneruj obraz tej sceny używając modelu cieniowania Gourauda. Użyj oryginalnych wektorów normalnych w każdym narożniku trójkątów. Do obliczania oświetlenia narożników zastosuj model Lamberta.

```
Im = uint8(zeros(Y,X));
```

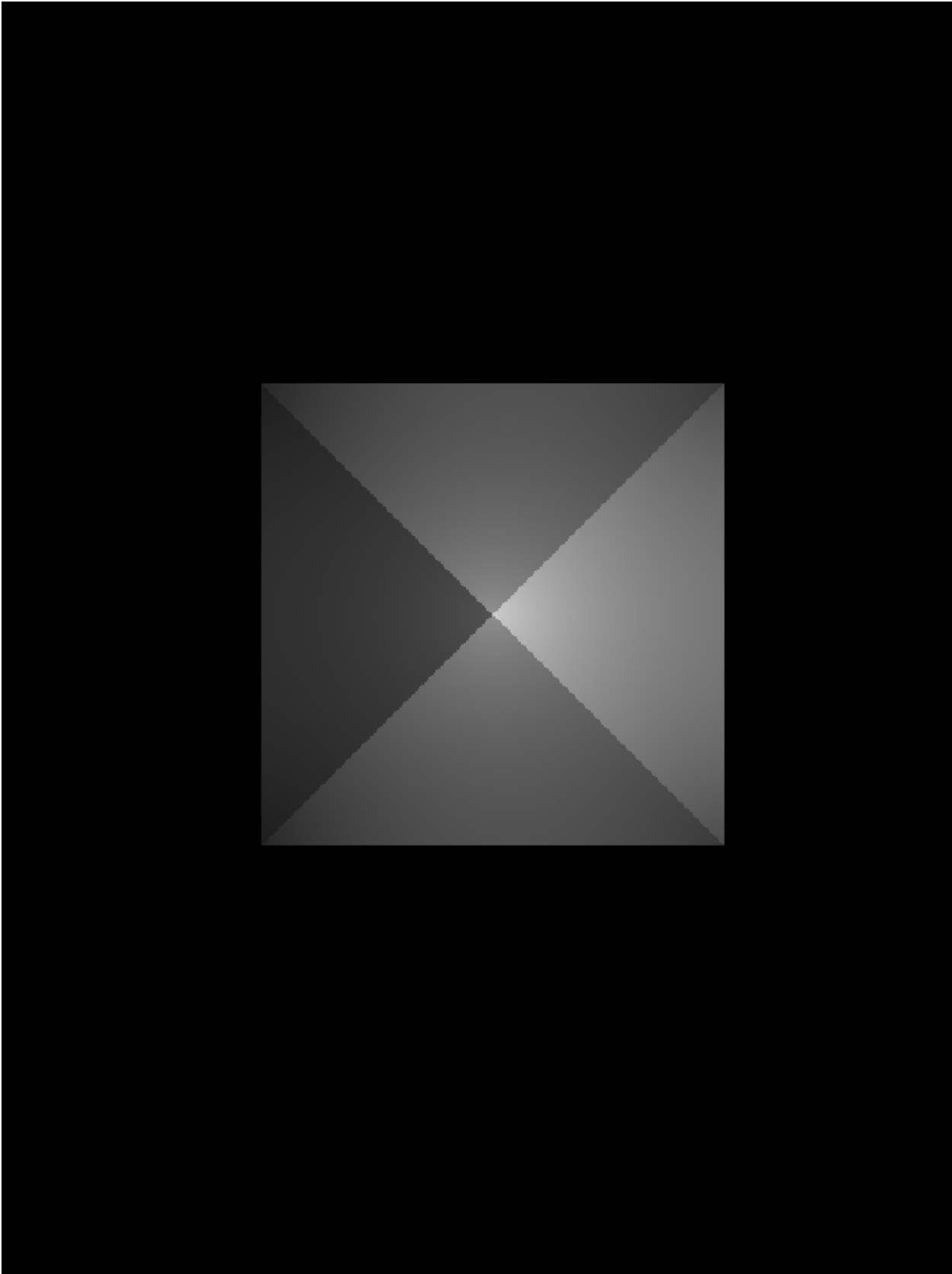
```
Im = gouraud_norm(Im, tri(1), amb, lamp, int, X, Y, dx, dy);
```

```
Im = gouraud_norm(Im, tri(2), amb, lamp, int, X, Y, dx, dy);
```

```
Im = gouraud_norm(Im, tri(3), amb, lamp, int, X, Y, dx, dy);
```

```
Im = gouraud_norm(Im, tri(4), amb, lamp, int, X, Y, dx, dy);
```

```
figure(4); imshow(Im);
```

ZADANIE 5

Wygeneruj obraz tej sceny używając modelu cieniowania Gourauda. W narożnikach użyj wektorów normalnych, które są średnią wektorów normalnych trójkątów stykających się w danym narożniku. Do obliczania oświetlenia narożników zastosuj model Lamberta.

```
Im = uint8(zeros(Y,X));

ave = zeros(3,5);
% 1: wierzchołek ostrosłupa, wspólny dla 4 trójkątów
ave(:,1) = ave_vect([tri(1).A, tri(1).B, tri(1).C, ...
                    tri(2).A, tri(2).B, tri(2).C, ...
                    tri(3).A, tri(3).B, tri(3).C, ...
                    tri(4).A, tri(4).B, tri(4).C], 4);

% 2: tri(1).B = tri(4).C
ave(:,2) = ave_vect([tri(1).A, tri(1).B, tri(1).C, ...
                    tri(4).A, tri(4).B, tri(4).C], 2);

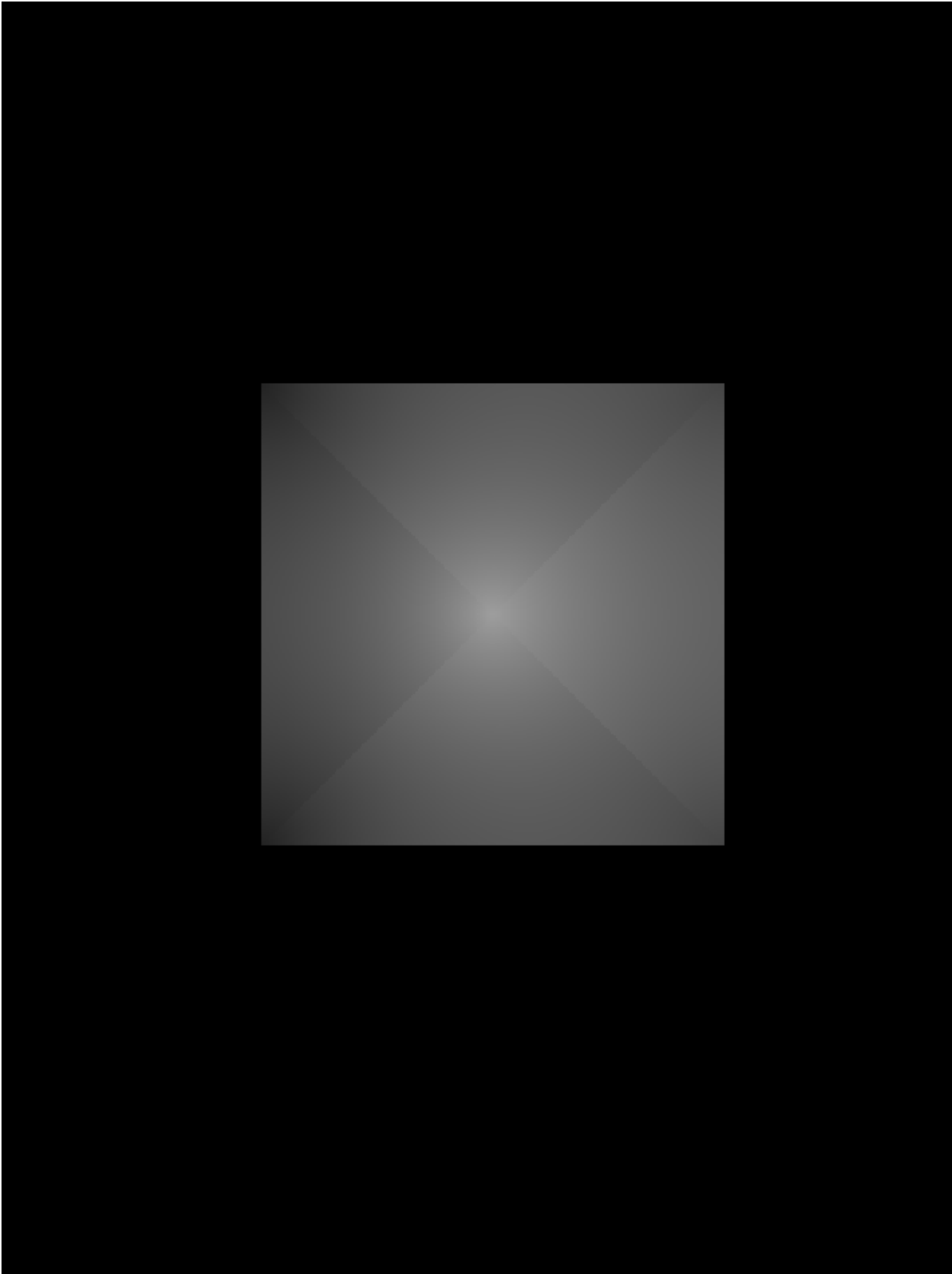
% 3: tri(1).C = tri(2).B
ave(:,3) = ave_vect([tri(1).A, tri(1).B, tri(1).C, ...
                    tri(2).A, tri(2).B, tri(2).C], 2);

% 4: tri(2).C = tri(3).B
ave(:,4) = ave_vect([tri(2).A, tri(2).B, tri(2).C, ...
                    tri(3).A, tri(3).B, tri(3).C], 2);

% 5: tri(3).C = tri(4).B
ave(:,5) = ave_vect([tri(3).A, tri(3).B, tri(3).C, ...
                    tri(4).A, tri(4).B, tri(4).C], 2);

Im = gouraud_spec(Im, tri(1), [ave(:,1), ave(:,2), ave(:,3)], amb, lamp, int,
X, Y, dx, dy);
Im = gouraud_spec(Im, tri(2), [ave(:,1), ave(:,3), ave(:,4)], amb, lamp, int,
X, Y, dx, dy);
Im = gouraud_spec(Im, tri(3), [ave(:,1), ave(:,4), ave(:,5)], amb, lamp, int,
X, Y, dx, dy);
Im = gouraud_spec(Im, tri(4), [ave(:,1), ave(:,5), ave(:,2)], amb, lamp, int,
X, Y, dx, dy);

figure(5); imshow(Im);
```



ZADANIE 6

Wygeneruj obraz tej sceny używając modelu cieniowania Phong. W narożnikach użyj wektorów normalnych, które są średnią wektorów normalnych trójkątów stykających się w danym narożniku. Do obliczania oświetlenia punktów (pixeli) zastosuj model Lamberta.

```
Im = uint8(zeros(Y,X));

Im = phong_spec(Im, tri(1), [ave(:,1), ave(:,2), ave(:,3)], amb, lamp, int,
X, Y, dx, dy);
Im = phong_spec(Im, tri(2), [ave(:,1), ave(:,3), ave(:,4)], amb, lamp, int,
X, Y, dx, dy);
Im = phong_spec(Im, tri(3), [ave(:,1), ave(:,4), ave(:,5)], amb, lamp, int,
X, Y, dx, dy);
Im = phong_spec(Im, tri(4), [ave(:,1), ave(:,5), ave(:,2)], amb, lamp, int,
X, Y, dx, dy);

figure(6); imshow(Im);
```

FUNKCJE POMOCNICZE

LAMBERT

```
function Im = lambert(Im, triangle, amb, lamp, int, X, Y, dx, dy)
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    Ish = floodfill(Ish, round((j1+j2+j3)/3), round((i1+i2+i3)/3), 0, 255); %
    wypełnienie trójkąta białym kolorem

    [A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie
    parametrów płaszczyzny trójkąta
    n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
    if n(3) > 0
        n = -1*n;
    end % Obrócenie normalnej, jeśli jest skierowana w dół

    for i = 1:X
        for j = 1:Y
            if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
                trójkąta
                    [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
                    współrzędnych pikselowych na matematyczne
                    [~, x, y, z] = dist_to_plane([A,B,C,D], xx, yy, 0, 0, 0, 1);
```

```

% Wyznaczenie punktu na płaszczyźnie trójkąta
    l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy
    dist = norm(l); % Odległość od lampy
    l = l / dist; % Normalizacja wektora l
    cs = n' * l;
    if cs < 0
        cs = 0;
    end % Obliczenie cosinusa kąta między normalną a wektorem do
lampy
    temp = amb + int * cs / (1 + 0.001 * dist^2); % Obliczenie
natężenia oświetlenia z uwzględnieniem tłumienia
    if temp > 255
        temp = 255;
    end % Ograniczenie natężenia do 255
    Im(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
trójkąta
end
end
end
end

```

PHONG

```

function Im = phong(Im, triangle, amb, lamp, int, X, Y, dx, dy, e, eye, m)
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    Ish = floodfill(Ish, round((j1+j2+j3)/3), round((i1+i2+i3)/3), 0, 255); %
wypełnienie trójkąta białym kolorem

    [A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie
parametrów płaszczyzny trójkąta
    n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
    if n(3) > 0
        n = -1*n;
    end % Obrócenie normalnej, jeśli jest skierowana w dół

    for i = 1:X
        for j = 1:Y
            if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
trójkąta
                [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
współrzędnych pikselowych na matematyczne
                [~, x, y, z] = dist_to_plane([A,B,C,D], xx, yy, 0, 0, 0, 1);
% Wyznaczenie punktu na płaszczyźnie trójkąta
                l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy

```

```

    dist = norm(l); % Odległość od lampy
    l = l / dist; % Normalizacja wektora l
    cs = n'* l;
    if cs < 0
        cs = 0;
    end % Obliczenie cosinusa kąta między normalną a wektorem do
lampy

    rp = 2 * (n'* l) * n - l; % Wektor odbicia
    rp = rp / norm(rp); % Normalizacja wektora odbicia
    vp = eye - [x; y; z]; % Wektor od punktu na trójkącie do oka
    vp = vp / norm(vp); % Normalizacja wektora do oka
    cs2 = rp'* vp;
    if cs2 < 0
        cs2 = 0;
    end % Obliczenie cosinusa kąta między wektorem odbicia a
wektorem do oka

    temp = amb + int * cs * e * (1 + (cs2^m)) / (1 + 0.001 *
dist^2); % Obliczenie natężenia oświetlenia z uwzględnieniem tłumienia
    if temp > 255
        temp = 255;
    end % Ograniczenie natężenia do 255 z uwzględnieniem tłumienia
    Im(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
trójkąta
end
end
end
end

```

CONST

```

function Im = const(Im, triangle, amb, lamp, int, X, Y, dx, dy)
    Ish = uint8(zeros(Y,X)); % pomocniczy obraz zawierający biały trójkąt

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    % srodek trojkata
    i0 = round((i1 + i2 + i3) / 3);
    j0 = round((j1 + j2 + j3) / 3);

    Ish = floodfill(Ish, j0, i0, 0, 255); % wypełnienie trójkąta białym
kolorem

    [A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie
parametrów płaszczyzny trójkąta

```

```

n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego
if n(3) > 0
    n = -1*n;
end % Obrócenie normalnej, jeśli jest skierowana w dół

[xx,yy] = pix_to_mat(i0, j0, X, Y, dx, dy);
[~, x, y, z] = dist_to_plane([A,B,C,D], xx, yy, 0, 0, 0, 1);
l = lamp - [x;y;z];
dist = norm(l);
l = l/dist;
cs = n'*l;
if cs < 0
    cs = 0;
end
temp = amb + int * cs / (1+0.001*dist*dist);
if temp > 255
    temp = 255;
end

for i = 1:X
    for j = 1:Y
        if Ish(j,i) == 255 % Sprawdzenie, czy piksel jest wewnątrz
trójkąta
            Im(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
trójkąta
        end
    end
end
end
end

```

GOURAUD

```

function Im = gouraud_norm(Im, triangle, amb, lamp, int, X, Y, dx, dy)
    Ish = uint8(zeros(Y,X));

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    % srodek trojkata
    i0 = round((i1 + i2 + i3) / 3);
    j0 = round((j1 + j2 + j3) / 3);

    Ish = floodfill(Ish, j0, i0, 0, 255); % wypełnienie trójkąta białym
kolorem

    [A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie
parametrów płaszczyzny trójkąta
    n = [A; B; C] / norm([A; B; C]); % Normalizacja wektora normalnego

```

```

if n(3) > 0
    n = -1*n;
end % Obrócenie normalnej, jeśli jest skierowana w dół

% kolejne narożniki

l = lamp-triangle.A;
dist = norm(l);
l = l/dist;
cs = n'*l;
if cs < 0
    cs = 0;
end
nar1 = amb + int * cs / (1+0.001*dist*dist);
if nar1 > 255
    nar1 = 255;
end
Im(j1,i1) = nar1;

l = lamp-triangle.B;
dist = norm(l);
l = l/dist;
cs = n'*l;
if cs < 0
    cs = 0;
end
nar2 = amb + int * cs / (1+0.001*dist*dist);
if nar2 > 255
    nar2 = 255;
end
Im(j2,i2) = nar2;

l = lamp-triangle.C;
dist = norm(l);
l = l/dist;
cs = n'*l;
if cs < 0
    cs = 0;
end
nar3 = amb + int * cs / (1+0.001*dist*dist);
if nar3 > 255
    nar3 = 255;
end
Im(j3,i3) = nar3;

for i = 1:X
    for j = 1:Y
        if Ish(j,i)==255
            if isequal([j,i],[j1,i1]) continue; end
            if isequal([j,i],[j2,i2]) continue; end
            if isequal([j,i],[j3,i3]) continue; end
            invd1 = 1/sqrt((j-j1)^2+(i-i1)^2); % współrzędne
barycentryczne

```

```

        invd2 = 1/sqrt((j-j2)^2+(i-i2)^2);
        invd3 = 1/sqrt((j-j3)^2+(i-i3)^2);
        Im(j,i) = (nar1*invd1+nar2*invd2+nar3*invd3)/
(invnd1+invnd2+invnd3);
    end
end
end
end
end

```

GOURAUD SPEC

```

function Im = gouraud_spec(Im, triangle, ave, amb, lamp, int, X, Y, dx, dy)
    Ish = uint8(zeros(Y,X));

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    % srodek trojkata
    i0 = round((i1 + i2 + i3) / 3);
    j0 = round((j1 + j2 + j3) / 3);

    Ish = floodfill(Ish, j0, i0, 0, 255); % wypełnienie trójkąta białym
    kolorem

    % kolejne narożniki
    l = lamp-triangle.A;
    dist = norm(l);
    l = l/dist;
    cs = ave(:,1)'*l;
    if cs < 0
        cs = 0;
    end
    nar1 = amb + int * cs / (1+0.001*dist*dist);
    if nar1 > 255
        nar1 = 255;
    end
    Im(j1,i1) = nar1;

    l = lamp-triangle.B;
    dist = norm(l);
    l = l/dist;
    cs = ave(:,2)'*l;
    if cs < 0
        cs = 0;
    end
    nar2 = amb + int * cs / (1+0.001*dist*dist);
    if nar2 > 255
        nar2 = 255;
    end

```

```

end
Im(j2,i2) = nar2;

l = lamp-triangle.C;
dist = norm(l);
l = l/dist;
cs = ave(:,3)'*l;
if cs < 0
    cs = 0;
end
nar3 = amb + int * cs / (1+0.001*dist*dist);
if nar3 > 255
    nar3 = 255;
end
Im(j3,i3) = nar3;

for i = 1:X
    for j = 1:Y
        if Ish(j,i)==255
            if isequal([j,i],[j1,i1]) continue; end
            if isequal([j,i],[j2,i2]) continue; end
            if isequal([j,i],[j3,i3]) continue; end
            invd1 = 1/sqrt((j-j1)^2+(i-i1)^2); % wspolrzedne
barycentryczne
            invd2 = 1/sqrt((j-j2)^2+(i-i2)^2);
            invd3 = 1/sqrt((j-j3)^2+(i-i3)^2);
            Im(j,i) = (nar1*invd1 + nar2*invd2 + nar3*invd3) /
(inv1+inv2+inv3);
        end
    end
end
end
end

```

PHONG SPEC

```

function Im = phong_spec(Im, triangle, ave, amb, lamp, int, X, Y, dx, dy)
    Ish = uint8(zeros(Y,X));

    [i1,j1] = mat_to_pix(triangle.A(1), triangle.A(2), X, Y, dx, dy);
    [i2,j2] = mat_to_pix(triangle.B(1), triangle.B(2), X, Y, dx, dy);
    [i3,j3] = mat_to_pix(triangle.C(1), triangle.C(2), X, Y, dx, dy);

    Ish = line(Ish, j1, i1, j2, i2, dx, dy, 255);
    Ish = line(Ish, j2, i2, j3, i3, dx, dy, 255);
    Ish = line(Ish, j3, i3, j1, i1, dx, dy, 255);

    % srodek trojkata
    i0 = round((i1 + i2 + i3) / 3);
    j0 = round((j1 + j2 + j3) / 3);

    Ish = floodfill(Ish, j0, i0, 0, 255); % wypełnienie trójkąta białym

```

kolorem

[A, B, C, D] = plane(triangle.A, triangle.B, triangle.C); % Wyznaczenie parametrów płaszczyzny trójkąta

```
% kolejne narożniki
l = lamp-triangle.A;
dist = norm(l);
l = l/dist;
cs = ave(:,1)'*l;
if cs < 0
    cs = 0;
end
nar1 = amb + int * cs / (1+0.001*dist*dist);
if nar1 > 255
    nar1 = 255;
end
Im(j1,i1) = nar1;

l = lamp-triangle.B;
dist = norm(l);
l = l/dist;
cs = ave(:,2)'*l;
if cs < 0
    cs = 0;
end
nar2 = amb + int * cs / (1+0.001*dist*dist);
if nar2 > 255
    nar2 = 255;
end
Im(j2,i2) = nar2;

l = lamp-triangle.C;
dist = norm(l);
l = l/dist;
cs = ave(:,3)'*l;
if cs < 0
    cs = 0;
end
nar3 = amb + int * cs / (1+0.001*dist*dist);
if nar3 > 255
    nar3 = 255;
end
Im(j3,i3) = nar3;

for i = 1:X
    for j = 1:Y
        if Ish(j,i)==255
            if isequal([j,i],[j1,i1]) continue; end
            if isequal([j,i],[j2,i2]) continue; end
            if isequal([j,i],[j3,i3]) continue; end
            invd1 = 1/sqrt((j-j1)^2+(i-i1)^2); % wspolrzedne
```

barycentryczne

```

    invd2 = 1/sqrt((j-j2)^2+(i-i2)^2);
    invd3 = 1/sqrt((j-j3)^2+(i-i3)^2);

    n = (ave(:,1)*invd1 + ave(:,2)*invd2 + ave(:,3)*invd3) /
(invnd1+invnd2+invnd3);

    [xx,yy] = pix_to_mat(i, j, X, Y, dx, dy); % Zamiana
współrzędnych pikselowych na matematyczne
    [~, x, y, z] = dist_to_plane([A,B,C,D], xx, yy, 0, 0, 0, 1);
% Wyznaczenie punktu na płaszczyźnie trójkąta
    l = lamp - [x; y; z]; % Wektor od punktu na trójkącie do lampy
    dist = norm(l); % Odległość od lampy
    l = l / dist; % Normalizacja wektora l
    cs = n'* l;
    if cs < 0
        cs = 0;
    end % Obliczenie cosinusa kąta między normalną a wektorem do
lampy
    temp = amb + int * cs / (1 + 0.001 * dist^2); % Obliczenie
natężenia oświetlenia z uwzględnieniem tłumienia
    if temp > 255
        temp = 255;
    end % Ograniczenie natężenia do 255 z uwzględnieniem tłumienia
    Im(j, i) = temp; % Ustawienie natężenia dla pikseli wewnątrz
trójkąta
end
end
end
end
end

```

AVE VECT

```

function ave = ave_vect(r,N)
% r: tablica narożników N stykających się trójkątów
ave = [0;0;0];
for i = 0:N-1
    r1(1:3) = r(1:3,3*i+1);
    r2(1:3) = r(1:3,3*i+2);
    r3(1:3) = r(1:3,3*i+3);
    [A, B, C, ~] = plane(r1,r2,r3);
    n = [A; B; C]/norm([A; B; C]); % wektor normalny
    if n(3)>0
        n = -1*n;
    end % skierowany we właściwą stronę
    ave = ave+n;
end
ave = ave/norm(ave); % normalizacja
end

```

Funkcja do zamiany współrzędnych matematycznych na pikselowe i odwrotnie

x, y - współrzędne M, N - szerokość i wysokość obrazu dx, dy - rozmiar pojedynczego piksela

```
function [i,j] = mat_to_pix(x, y, M, N, dx, dy)
    T = [1/dx, 0, M/2;
         0, -1/dy, N/2;
         0, 0, 1];
    temp = [x; y; 1];
    res = T * temp;
    i = round(res(1)/res(3));
    j = round(res(2)/res(3));
end
```

```
function [i,j] = pix_to_mat(x, y, M, N, dx, dy)
    T = [dx, 0, -M*dx/2;
         0, -dy, N*dy/2;
         0, 0, 1];
    res = T * [x; y; 1];
    i = res(1)/res(3);
    j = res(2)/res(3);
end
```

Funkcja rysuje odcinek na obrazie monochromatycznym.

```
function Im = line(Im, ya, xa, yb, xb, dx, dy, color)
    % Im - obraz
    % (xa, ya) - punkt początkowy
    % (xb, yb) - punkt końcowy
    % color - kolor linii

    [Y, X] = size(Im);

    if (abs(yb-ya) > abs(xb-xa)) % stroma linia
        % zamiana współrzędnych
        x0 = ya;
        y0 = xa;
        x1 = yb;
        y1 = xb;

        % zamiana rozmiarów X i Y
        temp = X; X = Y; Y = temp;

        zamiana = 1;
    else
        x0 = xa;
        y0 = ya;
        x1 = xb;
    end
```

```
    y1 = yb;

    zamiana = 0;
end

% zamiana poczatku i konca linii, zeby zaczynac od lewej
if(x0 > x1)
    temp1 = x0; x0 = x1; x1 = temp1;
    temp2 = y0; y0 = y1; y1 = temp2;
end

dx = abs(x1 - x0) ;           % odleglosc x
dy = abs(y1 - y0);           % odleglosc y
sy = sign(y1 - y0);           % znak przyrostu w kierunku y

x = x0; y = y0;               % inicjalizacja
if x > 0 && x <= X && y > 0 && y <= Y
    if (zamiana == 0)
        Im(y,x) = color;      % rysowanie punktu
    else
        Im(x,y) = color;
    end
end

error = 2*dy - dx ;           % inicjalizacja bledu
for i = 0:dx-1

    error = error + 2*dy;       % modyfikacja bledu
    if (error > 0)
        y = y + sy;
        error = error - 2*dx;
    end

    x = x + 1;                 % zwiekszamy x
    if x > 0 && x <= X && y > 0 && y <= Y
        if (zamiana == 0)
            Im(y,x) = color;    % rysowanie punktu
        else
            Im(x,y) = color;
        end
    end
end
end
```

Funkcja do wyznaczania parametrow plaszczyny $Ax+By+Cz+D=0$ przechodzacej przez trzy punkty p1,p2,p3

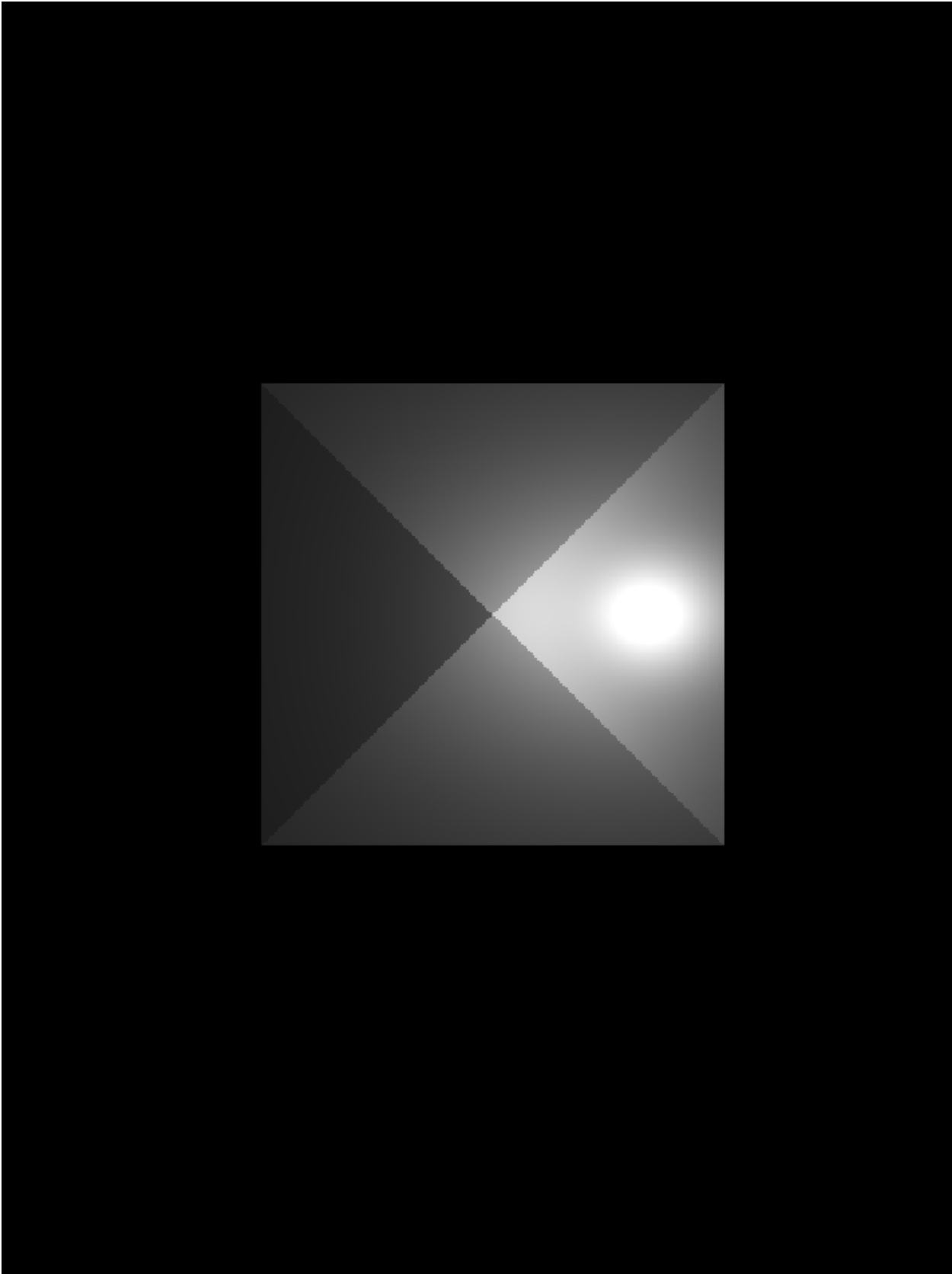
```
function [A, B, C, D] = plane(p1,p2,p3)
    A = det([p1(2),p1(3),1;
             p2(2),p2(3),1;
```

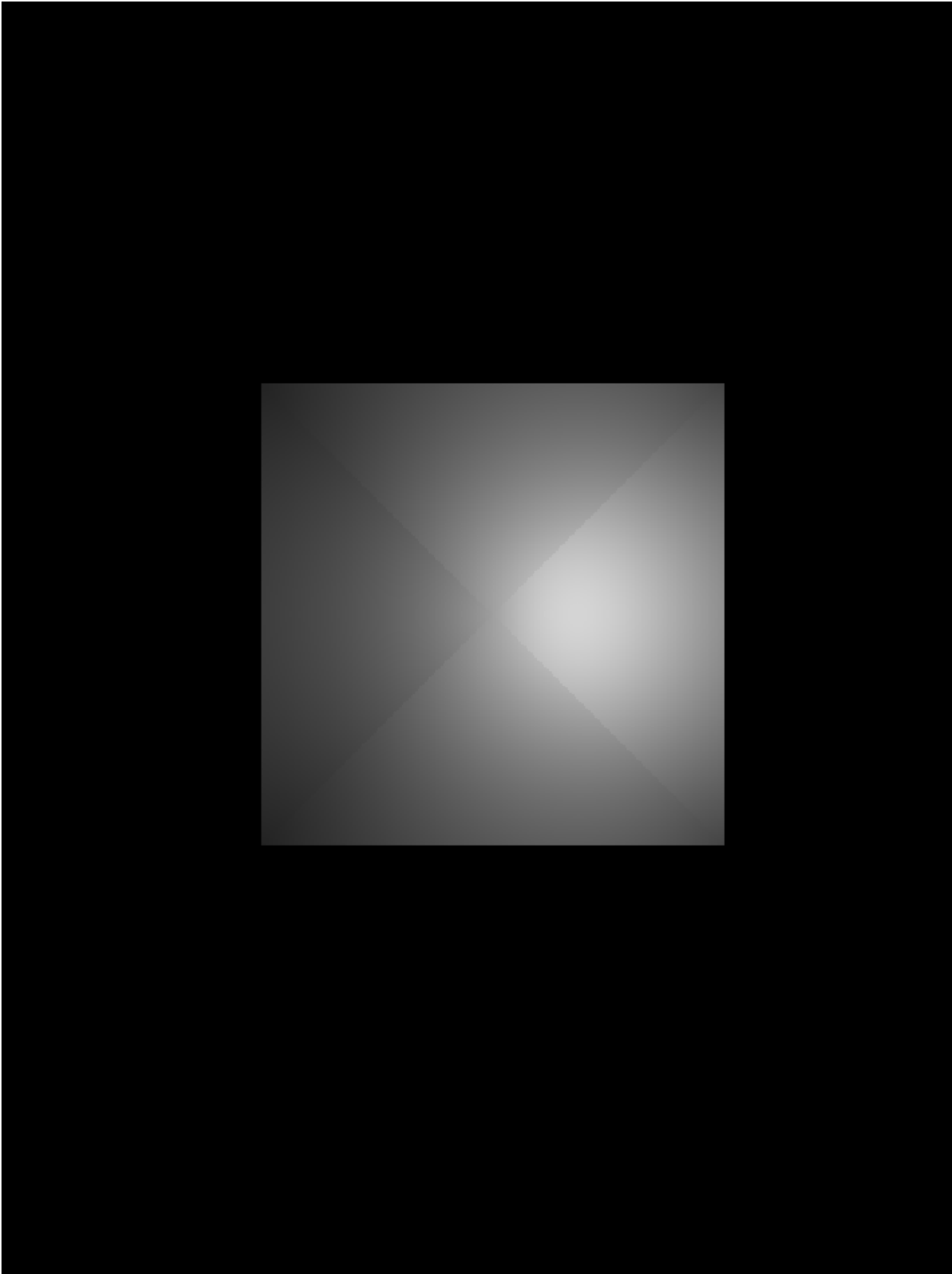
```
        p3(2), p3(3), 1]);  
B = -det([p1(1), p1(3), 1;  
        p2(1), p2(3), 1;  
        p3(1), p3(3), 1]);  
C = det([p1(1), p1(2), 1;  
        p2(1), p2(2), 1;  
        p3(1), p3(2), 1]);  
D = -det([p1(1), p1(2), p1(3);  
        p2(1), p2(2), p2(3);  
        p3(1), p3(2), p3(3)]);  
end
```

Funkcja do wyznaczania odleglosci punktu [x,y,z] od plaszczyzny o rownaniu $Ax+By+Cz+D=0$

odleglosc mierzona jest wzdluz prostej o wektorze kierunkowym [l,m,n] zwraca rowniez punkt przeciecia z plaszczyzna

```
function [d,x1,y1,z1] = dist_to_plane(plane, x, y, z, l, m, n)  
    ro = (plane(1)*x+plane(2)*y+plane(3)*z+plane(4))/  
    (plane(1)*l+plane(2)*m+plane(3)*n);  
    x1 = x-l*ro; y1 = y-m*ro; z1 = z-n*ro;  
    d = sqrt((x-x1)^2+(y-y1)^2+(z-z1)^2);  
end
```





Funkcja wypelnia zamkniety obszar obrazu metoda floodfill.

```
function Im = floodfill(Im, y, x, T, color)
    % Im - obraz
    % (y,x) - punkt startowy
    % T - jasnosć do zamiany
    % color - nowy kolor

    [Y, X] = size(Im);

    % sprawdzenie zakresu
    if y < 1 || y > Y || x < 1 || x > X
        error('Punkt startowy (y,x) jest poza obrazem.');
```

end

```
    % sprawdzenie jasności piksela startowego
    if Im(y,x) ~= T
        %disp('Piksel startowy ma zła jasność.');
```

return;

```
    % w naszym kodzie oznacza to że trafiliśmy na krawędź - a więc
    % trójkąt jest obrzucony do nas bokiem.
    end

    % jeśli nowa jasność taka sama jak stara, nic nie trzeba robić
    if T == color
        return;
    end

    % kolejka pikseli do odwiedzenia
    Q = zeros(Y*X, 2);
    head = 1;
    tail = 1;

    Q(tail,:) = [y, x];
    Im(y,x) = color;

    while head <= tail
        cy = Q(head,1);
        cx = Q(head,2);
        head = head + 1;

        % 4-sąsiedztwo: góra, dół, lewo, prawo

        % góra
        ny = cy - 1;
        nx = cx;
        if ny >= 1 && Im(ny,nx) == T
            tail = tail + 1;
            Q(tail,:) = [ny, nx];
            Im(ny,nx) = color;
        end
```

```
% dol
ny = cy + 1;
nx = cx;
if ny <= Y && Im(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im(ny,nx) = color;
end

% lewo
ny = cy;
nx = cx - 1;
if nx >= 1 && Im(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im(ny,nx) = color;
end

% prawo
ny = cy;
nx = cx + 1;
if nx <= X && Im(ny,nx) == T
    tail = tail + 1;
    Q(tail,:) = [ny, nx];
    Im(ny,nx) = color;
end
end
end
```

Published with MATLAB® R2026a